

Module-1

Introduction to COA

* RISC v/s CISC

RISC

- RISC stands for Reduced Instⁿ set computing
- RISC uses smaller set of instruction being simple.
- The execution speed is faster. (single cycle)
- Instⁿ length is fixed.
- The power efficiency is generally more efficient.
- Ex - Power PC, RISC-V

CISC.

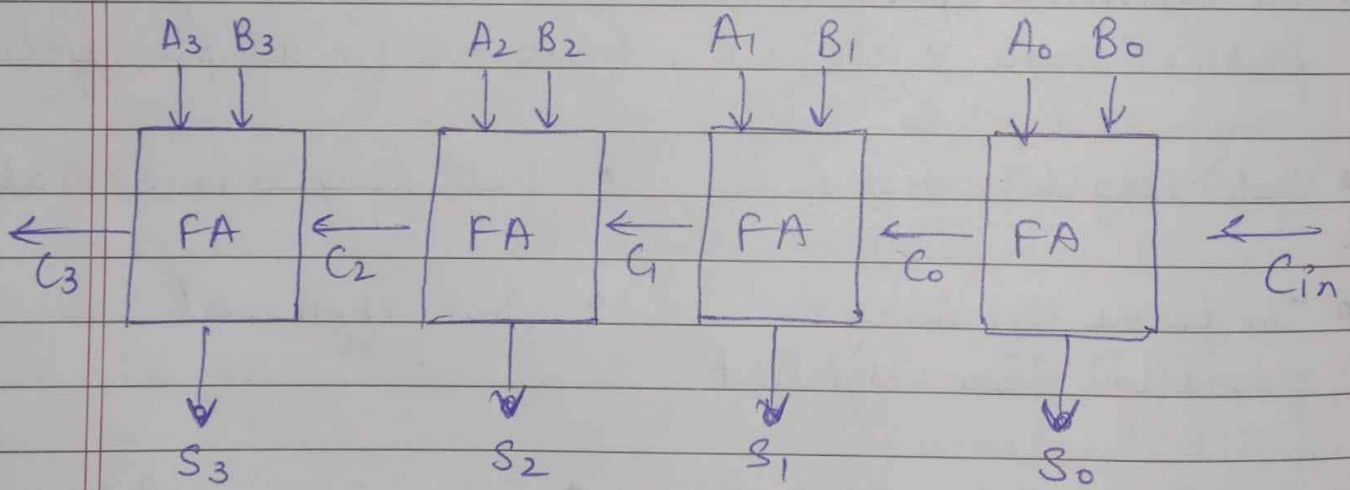
- CISC stands for Complex Instⁿ set Computing.
- CISC uses large set of instrⁿ set, more complex instⁿ set.
- The execution speed is slower (multiple cycle)
- Instⁿ length is variable.
- Less efficient.
- Ex - x86, VAX.

* Ripple Carry Adder. :

- A RCA is a digital circuit that performs binary addition by adding two numbers bit by bit.
- It is characterized by the 'ripping' or carry signals from the least significant bit (LSB) to the most

significant bit (MSB). This means that the carry output of one full adder stage becomes the carry input of the next.

- A RCA is built by connecting multiple full adder blocks in series.
- Each full adder adds two bits of the numbers being added and a carry-in from the previous stage.
- It generates a sum bit and carry out bit.
- RCA is relatively easy to design & implement.



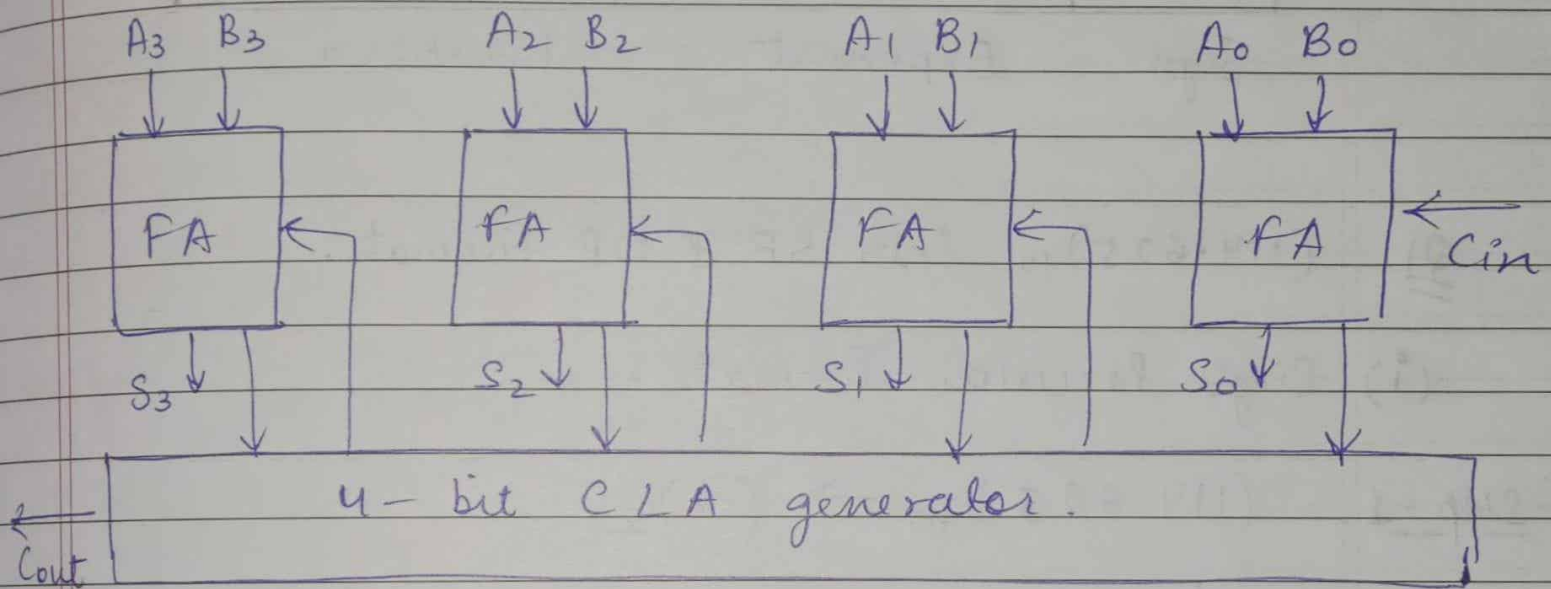
4-bit RCA.

* Carry Look Ahead Adder :

- Carry look ahead adder is the fastest adder circuit.
- It reduces the propagation delay, which occurs

during addition, by using more complex hardware circuitry.

- It is designed by transforming the RCA circuit such that carry logic of the adder is changed into two level logic.



* Single Precision & Double Precision format :

i) Single Precision Format (SP)

$\pm 1. BBB \times 2^{\pm \text{exp}}$

↑ ↑ ↑

Sign Fraction Exponent

1 bit	8 bit	23 bit
Sign	Exponent	Mantissa
(S)	(E)	(M)

Q12) Double Precision format :

$$\begin{array}{ccccccc} \pm & 1 & . & B & B & B & \times 2^{\pm \text{Exp.}} \\ \uparrow & & & \uparrow & & & \uparrow \\ \text{Sign} & & & \text{fraction} & & & \text{Exponent} \end{array}$$

1 bit	11 bit	52 bit
Sign	Exponent	Mantissa

Q1 $(114.625)_{10}$ into SP & DP format.

(i) Single Precision format :

Step-1 $(114.625)_{10} \rightarrow (2)_2$

2	114
2	57 - 0
2	56 - 1
2	28 - 0
2	14 - 0
2	7 - 0
2	3 - 1
	1 - 1

$$\begin{aligned} 0.625 \times 2 &= 1.25 - 1 \\ 0.25 \times 2 &= 0.50 - 0 \\ 0.50 \times 2 &= 1.00 - 1 \end{aligned}$$

$$(11100010.101)_2$$

Step-2 Normalize :

$$1. \underbrace{1100010101}_M \times 2^6$$

$$1. M \times 2^6$$

$$E - 127 = 6$$

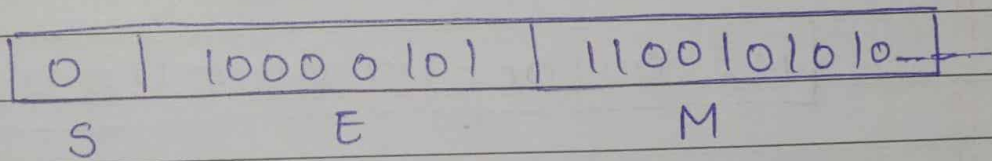
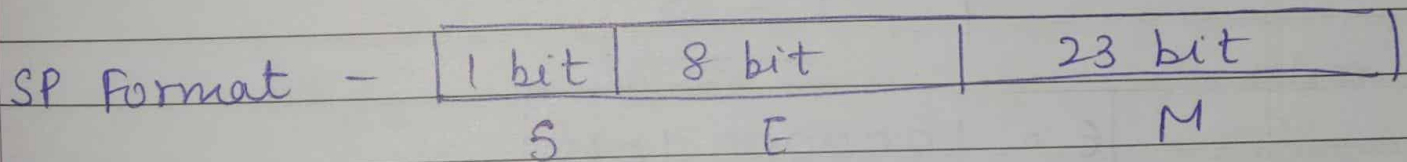
$$E = 127 + 6$$

$$E = 133$$

2	133
2	66 - 1
2	33 - 0
2	16 - 1
2	8 - 0
2	4 - 0
2	2 - 0
2	1 - 0

$$E = 133 = (10000101)_2$$

$$M = 110010101$$



ii) Double Precision Format:

$$(114.625)_{10} \rightarrow (x)_2$$

$$(000100010100.010000100101)_2$$

step-2 Normaliz E =

$$0.\underline{00100010100}01100010101 \times 2^{11}$$

M

$$E - 1023 = 11$$

$$0.M \times 2^{11}$$

$$E = 1023 + 11$$

$$E = 1034$$

2	1034
2	517 - 0
2	258 - 1
2	129 - 0
2	64 - 1
2	32 - 0
2	16 - 0
2	8 - 0
2	4 - 0
2	2 - 0
	1 - 0

$$E = 10000001010$$

$$M = 0010001010001100010101$$

DP format:

1 bit	11 bit	52 bit
S	E	M

⇒

0	10000001010	1001000101000110001010
S	E	M

* Restoring & Non Restoring Division Algo :

Restoring

- Restoring division is an integral procedure employed when calculating division on unsigned number.

The speed is slower due to restore operation.

Simpler logic and easy to implement.

It is used in older or simpler hardware.

Non - Restoring

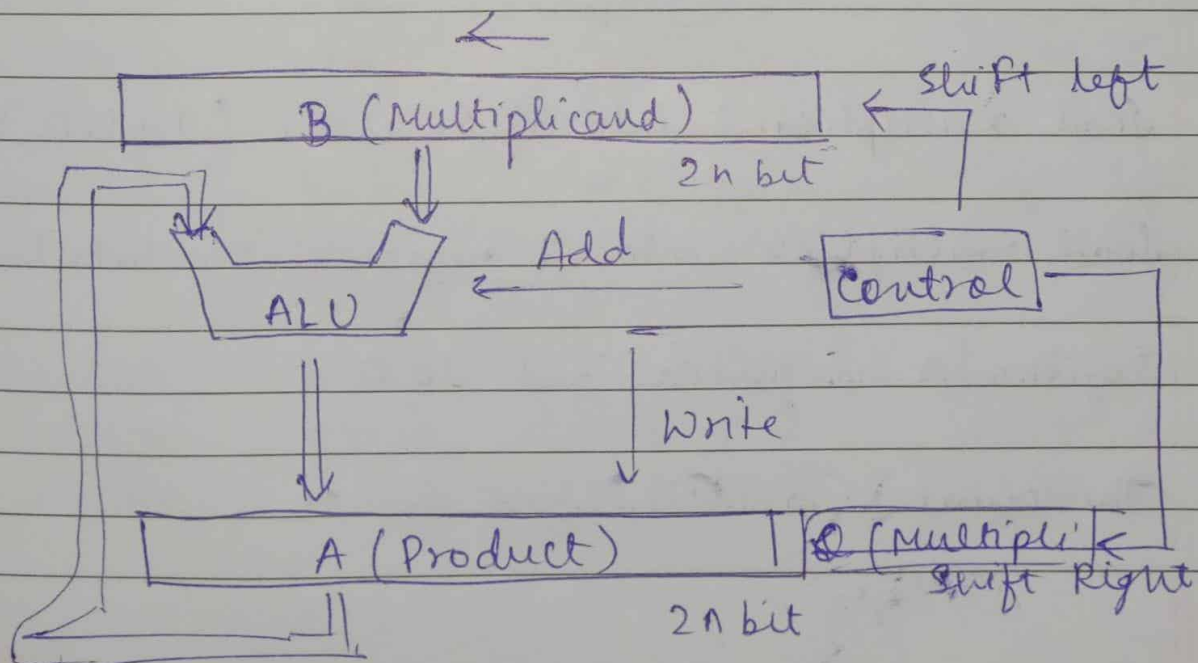
- It is a technique for unsigned binary values that simplifies the procedure by eliminating the restoring phase.

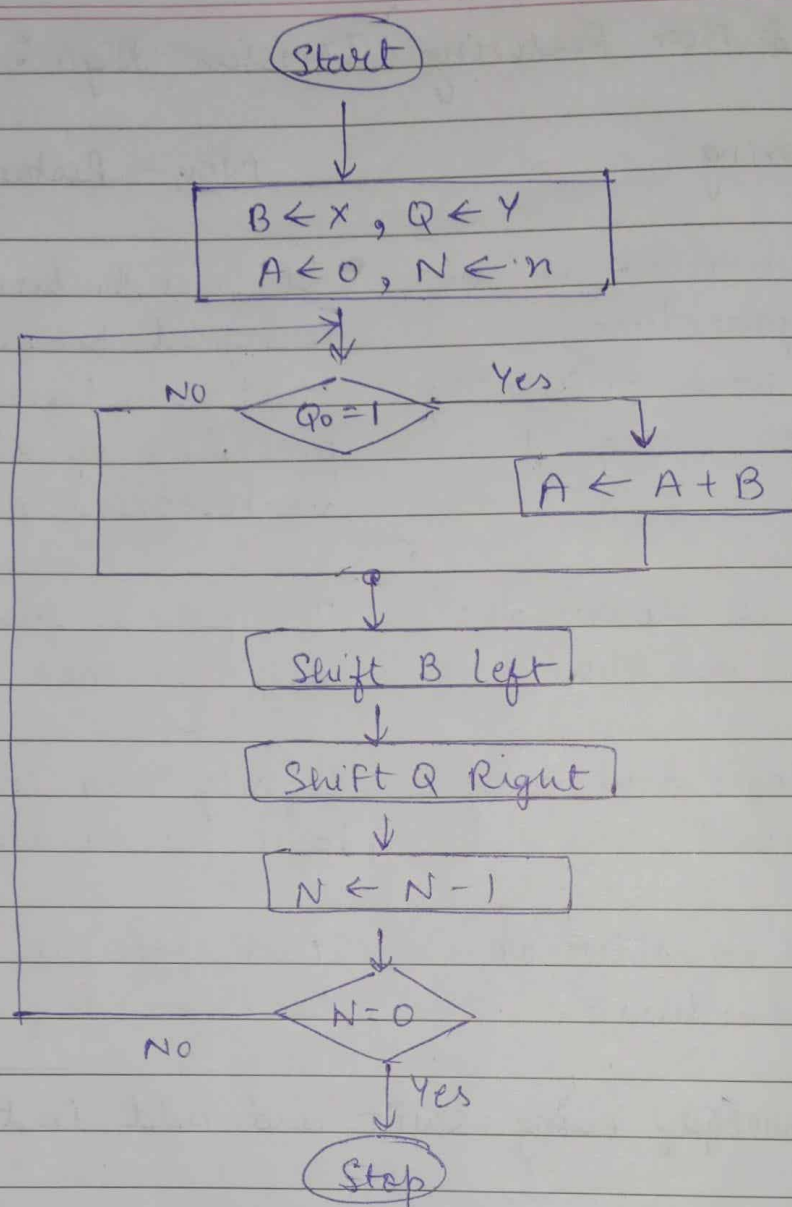
The speed is faster as it skips restore step.

Slightly more complex logic for handling design.

More efficient in modern hardware implementation.

* How to multiply using shift and add instruction?





Step-1 Load multiplicand into B register ($2n$ bits)

Step-2 Load multiplier into Q register (n bits)

Step-3 Register A is initialized to 0.

Step 4 The count N is initialized to n .

- S-5) The LSB of multiplier register (Q_0) determines whether the multiplicand is added to product register.
- S-6) The left shift of the multiplicand has the effect of shifting the intermediate product to the left.
- S-7) The right shift of the multiplier prepares the next bit of multiplier to examine the $Q_0 = 1$ condition & repeat the steps in flow chart.

Q1 Using 4-bit number, perform multiplication

$$\begin{array}{r} 9 \times 12 \\ \hline 9 \times 12 \\ (1001 \times 1100) \end{array}$$

Multiplier: 12
 Multiplicand: 9

A - Product
 Q - Multiplier
 B - Multiplicand

Step	A	Q	B	Operation
0	0000 0000	1100	0000 1001	Initialization
1	0000 0000	1100	000 10010	Shift left B
	0000 0000	0110	000 10010	Shift Right Q
2	0000 0000	0110	0010 0100	Shift left B
	0000 0000	0011	0010 0100	Shift Right Q
3	0010 0100	0011	0010 0100	Add B to A
	0010 0100	0011	0100 1000	Shift left B
	0010 0100	0001	0100 1000	Shift Right Q

4	0110 1100	0001	0100 1000	Add
	0110 1100	0001	1001 0000	Shift B to A
	0110 1100	0000	1001 0000	Shift left B
				Shift Right Q

★ Booth Algorithm :

1) Multiplication :

Start

Multiplicand $\rightarrow$ BR
Multiplier $\rightarrow$ QR

$AC \leftarrow 0$
 $Q_{n+1} \leftarrow 0$
 $SC \leftarrow n$

Q_n, Q_{n+1}

$= 10$

$= 01$

$AC \leftarrow AC + \overline{BR} + 1$

$AC \leftarrow AC + BR$

$= 00$
 $= 11$

Shift right (A||Q||R)
 $SC \leftarrow SC - 1$

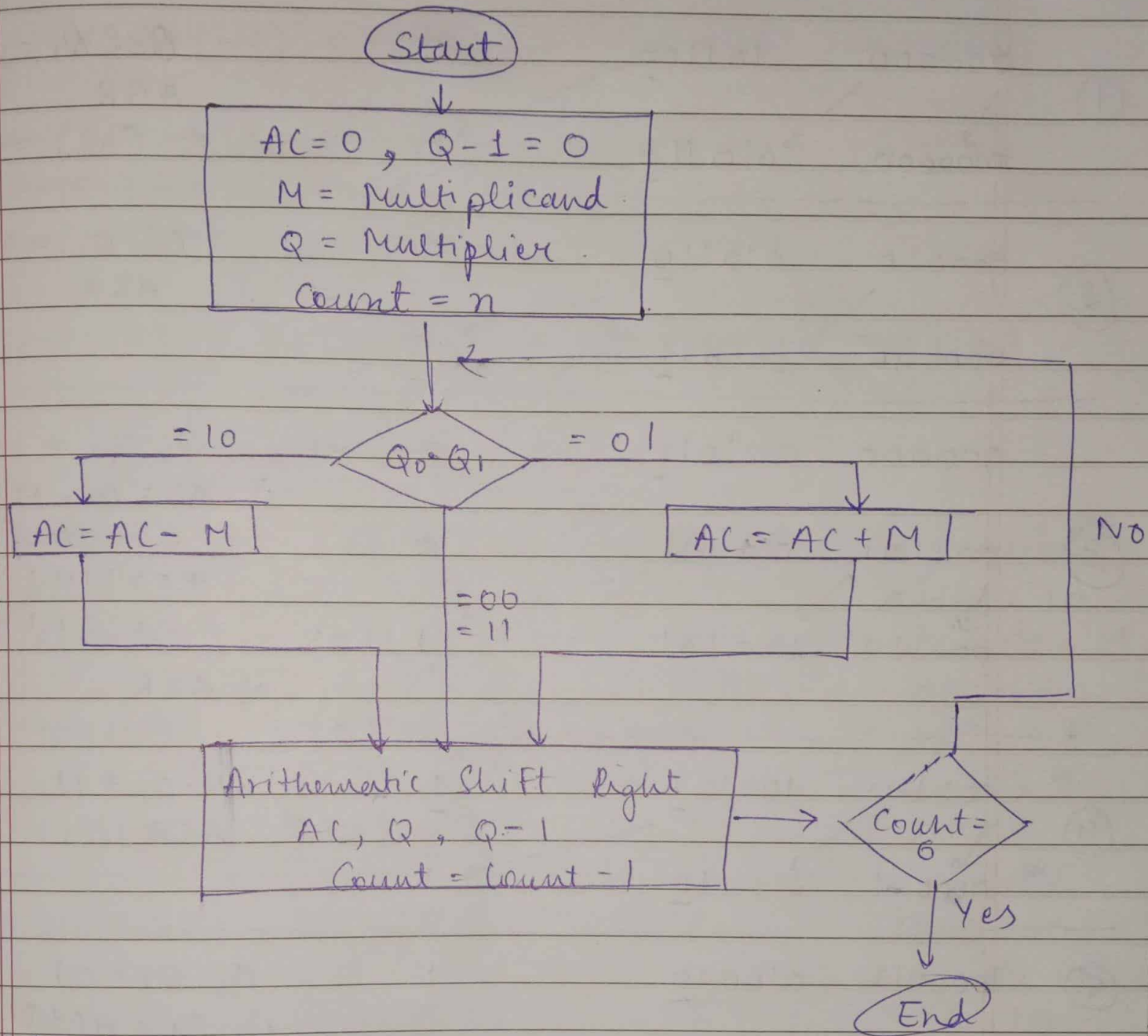
$\neq 0$

$= 0$

SC

End

* Booth Algorithm for Multiplication:



(i) $(-13)_{10} \times (-20)_{10}$

$$M = (-13)_{10} \rightarrow (110011)_2 \text{ 1's}$$

$$-M = (13)_{10} \rightarrow (001101)_2 \xrightarrow{2's} (110010)_2 \xrightarrow{2's} 110011$$

$$Q = (-20)_{10} \rightarrow (101100)_2 \text{ 1's}$$

$$-Q = (20)_{10} \rightarrow (010100)_2 \xrightarrow{2's} 101011 \xrightarrow{2's} 101011 + 1$$

$$\underline{\hspace{1cm}} \\ 101100$$

Step	AC	Q ₀	Q ₁	Operation
①	000000 ↓ 000000	101100 ↓ 010110	0 ↓ 0	Q ₀ · Q ₁ = 00 ASR

②	000000 ↓ 000000	010110 ↓ 001011	0 ↓ 0	Q ₀ · Q ₁ = 00 ASR
---	-----------------------	-----------------------	-------------	---

③	000000 ↓ 001101 ↓ 000110	001011 ↓ 001011 ↓ 100101	0 ↓ 0 ↓ 1	Q ₀ · Q ₁ = 10 i) AC = AC - M 000000 + 001101 001101 ii) ASR
---	--------------------------------------	--------------------------------------	-----------------------	---

④	000110 ↓ 000011	100101 ↓ 010010	1 ↓ 1	Q ₀ · Q ₁ = 11 ASR
---	-----------------------	-----------------------	-------------	---

⑤	000011 ↓ 110110 ↓ 111011	010010 ↓ 010010 ↓ 001001	1 ↓ 1 ↓ 0	Q ₀ · Q ₁ = 01 i) AC = AC + M 000011 + 110011 110110 ii) ASR
---	--------------------------------------	--------------------------------------	-----------------------	---

⑥	001000 ↓ 000100	001001 ↓ 000100	0 ↓ 1	Q ₀ · Q ₁ = 10 i) AC = AC - M 111011 + 001101 110110 ii) ASR
---	-----------------------	-----------------------	-------------	---

2) Multiply $(20)_{10}$ and $(-19)_{10}$ using Booth Algo?

$$(20) \times (-19) = -380$$

$$M \rightarrow (20) \rightarrow (010100)_2$$

$$-M \rightarrow -(20) \rightarrow (101100)_2$$

$$Q \rightarrow (-19) \rightarrow (101101)_2$$

$$-Q \rightarrow (19) \rightarrow (010011)_2 \xrightarrow{1's} (101100)_2 \xrightarrow{2's} 101101$$

Count = 6

Step	AC	Q	Q ₁	Operation
	000000	10110 <u>1</u>	<u>0</u>	$Q_0 \cdot Q_1 = 10$ 1) $AC = AC - M$
①	101100	101101	0	$\begin{array}{r} 000000 \\ + 101100 \\ \hline 101100 \end{array}$
	110100	01011 <u>0</u>	<u>1</u>	2) ASR
	110110	01011 <u>0</u>	<u>1</u>	$Q_0 \cdot Q_1 = 01$
②	001010	010110	1	1) $AC = AC + M$
	000101	00101 <u>1</u>	<u>0</u>	$\begin{array}{r} 110110 \\ + 010100 \\ \hline 001010 \end{array}$
				2) ASR

③ $000101 \quad 001011 \quad 0$ 1) $Q_0 Q_1 = 10$
 $AC = AC - M$
 000101
 $+ \underline{001100}$
 $\underline{010001}$
 2) ASR

Diagram for subtraction:
 000101
 $\swarrow \searrow$
 001011
 $\swarrow \searrow$
 $001000 \quad 100101$

④ $111000 \quad 100101 \quad 1$ 1) $Q_0 Q_1 = 11$
 ASR

Diagram for right shift:
 111000
 $\swarrow \searrow$
 $111100 \quad 010010$

⑤ $111100 \quad 010010 \quad 1$ 2) $Q_0 Q_1 = 01$
 $AC = AC + M$
 111100
 $+ \underline{010100}$
 $\underline{1010000}$
 2) ASR

Diagram for addition:
 111100
 $\swarrow \searrow$
 $010000 \quad 010010$
 $\swarrow \searrow$
 $001000 \quad 001001$

⑥ $001000 \quad 001001 \quad 0$ $Q_0 Q_1 = 10$
 $AC = AC - M$
 001000
 $+ \underline{101100}$
 $\underline{110100}$
 2) ASR

Diagram for subtraction:
 001000
 $\swarrow \searrow$
 001001
 $\swarrow \searrow$
 $110100 \quad 000100$

$(111010000100)_2$
 $\downarrow 1's$
 $000101111011 \xrightarrow{2's} 000101111100$

YMCA PYQs.

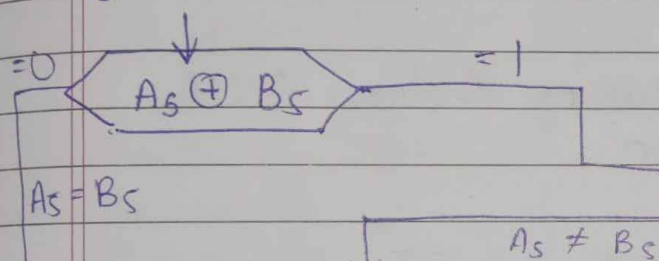
1) Draw the flowchart of addition & subtraction integer of binary no.

Subtraction

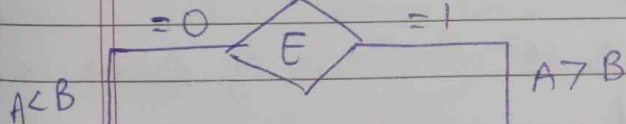


Minuend - A

Subtrahend - B



$E_p \leftarrow A + \bar{B} + 1$
 $AVF \leftarrow 0$



$A \leftarrow \bar{A}$

$A \leftarrow A + 1$
 $A_s \leftarrow \bar{A}_s$

$A_s \leftarrow 0$

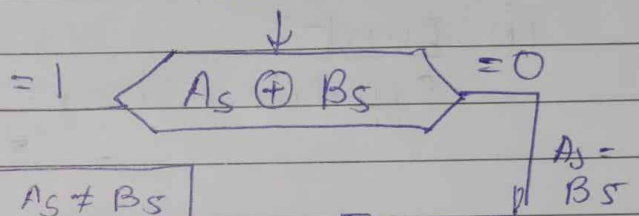
Result in A & A_s

Addition.



Augend - A

Addend - B



$E_A \leftarrow A + B$

$AVF \leftarrow E$

2) An address is stored at location 800 with the address field at location 801. The address field has the value of 900. A process or register Rx contains the number 800. Evaluate the effective address of the addressing mode of instⁿ is

i) direct

iii) Relative

ii) Immediate

iv) Register Indirect

(i) Direct :

$$\text{Effective Address} = \text{Value of Address field} \\ = 900$$

(ii) Immediate : 801

$$\text{Eff. Add} = \text{Address of Add. field} \\ = 801$$

(iii) Relative :

$$\text{Effective Address} = \text{Next Inst}^n + \text{Value of Address field}$$

$$[\because \text{Next Inst}^n \text{ Add} = \text{Current Inst}^n \text{ Add} + 1]$$

$$\text{Next Inst}^n \text{ Add} = 800 + 1 \Rightarrow 801$$

$$\therefore \text{Effective Add} = 801 + 900 = 1701$$

(iv) Register Indirect :

$$\text{Effective Add} = \text{Value in Register Rx} \\ = 800$$

3) An instⁿ is stored at location 300 with its add. field at location 301. The add. field has the value of 400. A processor register R₁ contains the no. 200. Evaluate the effective add. if add. mode of instⁿ is :

1) Direct

3) Relative

2) Immediate

4) Indirect

i) Direct :

$$\begin{aligned}\text{Effective Add} &= \text{Value of Add field} \\ &= 400\end{aligned}$$

ii) Immediate :

$$\begin{aligned}\text{EA} &= \text{Add of Add field} \\ &= 301\end{aligned}$$

iii) Relative :

$$\begin{aligned}\text{EA} &= \text{Next Inst}^n + \text{value of Add field} \\ &= 302 + 400 \\ &= 702\end{aligned}$$

iv) Indirect :

$$\text{EA} = 400$$

v) Register Indirect :

$$\begin{aligned}\text{Eff. Add} &= \text{value in Register R}_1 \\ &= 200.\end{aligned}$$

vi) Indexed Add :

$$\begin{aligned}\text{EA} &= \text{value in Register R}_1 + \text{value of Add field} \\ &= 200 + 400 \\ &= 600.\end{aligned}$$